

**Engineering, Built Environment
and IT**
Department of Computer Science
Programming Languages
COS 333
Semester Test 1
10 April 2024

Examiner: Mr W. S. van Heerden

Instructions:

Read the questions carefully and answer all questions.

This section comprises of **23** questions, and a total of **35** marks.

You have **1.5** hours to complete this test.

This test is an *online* assessment:

The test will automatically submit when the time (1.5 hours) expires.

You can also submit the test yourself if you are done ahead of time.

This test is **closed book** and is subject to the University of Pretoria integrity statement provided below.

You are not allowed to consult any sources other than the Practical 2 specification and the MIT/GNU Scheme Reference Manual (provided as documentation for Practical 2).

For practical implementation questions, your Scheme submission must be working Scheme code that will successfully interpret using version 8.8 of the DrRacket interpreter using the sicp language collections. You may (and should) use this interpreter to test your submissions. Once you have finished writing each program, make sure each is saved in a text file with the appropriate name (described in the question), and submit it to the correct upload slot.

You may use a non-programmable calculator.

You may not use any other programmable devices.

Integrity Statement:

The University of Pretoria commits itself to producing academic work of integrity. I affirm that I am aware of and have read the Rules and Policies of the University, more specifically the Disciplinary Procedure and the Tests and Examinations Rules, which prohibit any unethical, dishonest or improper conduct during tests, assignments, examinations and/or any other forms of assessment. I am aware that no student or any other person may assist or attempt to assist another student, or obtain help, or attempt to obtain help from another student or any other person during tests, assessments, assignments, examinations and/or any other forms of assessment.

Timed Test

This test has a time limit of 1 hour and 30 minutes. This test will save and be submitted automatically when the time expires.

Warnings appear when **half the time, 5 minutes, 1 minute, and 30 seconds** remain. *[The timer does not appear when previewing this test]*

Multiple Attempts

Not allowed. This Test can only be taken once.

Force Completion

This Test can be saved and resumed at any point until time has expired. The timer will continue to run if you leave the test.

QUESTION 1

1. Imagine that a senior developer has worked at a company for many years, but does not manage projects. The developer was originally trained by the company itself to use the C programming language, and has thus far also only used C as a development language on projects. A decision is made by the manager of the developer's team to use C# on the next project, which will start in a few months. **Identify** the most important reason for studying concepts of programming languages, which is applicable to the developer. You do not need to justify your answer.

1 points

QUESTION 2

1. Assume two hypothetical programming languages, A and B, both of which support primitive numeric values. Programming language A supports single dimensional arrays and two dimensional arrays as two separate data types, where single dimensional arrays and two dimensional arrays can both store primitive numeric values. Programming language B supports only single dimensional arrays, which can store primitive numeric values as well as other single dimensional arrays. Consider the following statements, and **select** the one that is the most correct.
- ☐ Programming language A is more orthogonal than programming language B, primarily because programming language A allows two dimensional arrays to be represented while programming language B does not.
 - ☐ Programming language A is more orthogonal than programming language B, primarily because more combinations of constructs are legal in programming language A.
 - ☐ Programming language B is more orthogonal than programming language A, primarily because more combinations of constructs are legal in programming language B.
 - ☐ Programming language B is more orthogonal than programming language A, primarily because programming language B has fewer constructs than programming language A.

1 points

QUESTION 3

1. Consider a hypothetical programming language that does not have a separate Boolean data type. Consider the following statements, and **select** the one that is the most correct.
- ☐ This language has reduced readability because a Boolean value cannot be represented.
 - ☐ This language has reduced readability because a Boolean value needs to be represented in a less natural way.
 - ☐ This language has reduced writability because a Boolean value cannot be represented.
 - ☐ This language has reduced writability because a Boolean value needs to be represented in a less natural way.

1 points

QUESTION 4

1. Compare imperative programming languages that support object-oriented programming and imperative programming languages that do not support object-oriented programming. Consider the following statements, and **select** the one that is the most correct.
- ☐ Imperative programming languages that support object-oriented programming can be considered less readable than imperative programming languages that do not support object-oriented programming.
 - ☐ Imperative programming languages that support object-oriented programming can be considered more readable than imperative programming languages that do not support object-oriented programming.
 - ☐ Imperative programming languages that support object-oriented programming can be considered less writable than imperative programming languages that do not support object-oriented programming.
 - ☐ Imperative programming languages that support object-oriented programming can be considered exactly as writable as imperative programming languages that do not support object-oriented programming.

1 points

QUESTION 5

1. Many modern programming languages have good support for type checking, while some older programming languages either did not support type checking or only partially supported type checking. Generally better type checking is considered to benefit a programming language. Consider the following statements about potential disadvantages associated with better type checking, and **select** the one that is the most correct.
- ☐ Good support for type checking will negatively impact the readability of a programming language.
 - ☐ Good support for type checking will negatively impact the reliability of a programming language.
 - ☐ Good support for type checking will negatively impact the cost of maintenance of a programming language.
 - ☐ Good support for type checking will negatively impact the execution cost of a programming language.

1 points

QUESTION 6

1. Consider the following contributing factors to the cost of a programming language or programming language feature, and **select** the most important one in a modern context.
- ☐ Cost of training programmers to use the language.
 - ☐ Cost of reliability.
 - ☐ Cost of compiling programs.
 - ☐ Cost of executing programs.

1 points

QUESTION 7

1. An additional programming language evaluation criterion that is not focussed on heavily in this course is the well-definedness of the programming language in question. Consider the following programming domains, and **select** the one that benefits the most from improved well-definedness in a programming language.
- ☐ Business applications.
 - ☐ Artificial intelligence.
 - ☐ Systems programming.
 - ☐ Web software.

1 points

QUESTION 8

1. Various factors can influence programming language design. Consider the following influences, and **select** the one that had the greatest impact on the programming language design for the earliest programming languages for scientific applications.
- ☐ Computer architecture.
 - ☐ Programmer efficiency becoming more important than machine efficiency.
 - ☐ The move from process oriented programming to data oriented programming.
 - ☐ The move to object-oriented programming.

1 points

QUESTION 9

1. Consider programming languages in which portability is very important, and **select** only the implementation methods that are well suited for these types of programming languages. Note that incorrectly selected options will be penalised.
- ☐ Compilation.
 - ☐ Pure interpretation.
 - ☐ Hybrid implementation systems.
 - ☐ No current implementation system is well suited for portability.

2 points

QUESTION 10

1. Consider the following statements about the Netbeans and UNIX programming environments, and **select** the one that is the most correct.
- ☐ The Netbeans programming environment is less integrated than the UNIX programming environment.
 - ☐ The Netbeans programming environment is less user friendly for most programmers than the UNIX programming environment.
 - ☐ The Netbeans programming environment is less time consuming for most programmers to use than the UNIX programming environment.
 - ☐ The Netbeans programming environment is less visual than the UNIX programming environment.

1 points

QUESTION 11

1. Compare Plankalkül and Fortran I (the first implemented version of Fortran). Consider the following statements, and **select** the one that is most correct.
- ☐ Plankalkül was more writable than Fortran I.
 - ☐ Plankalkül was less readable than Fortran I.
 - ☐ Plankalkül had better support for data structures than Fortran I.
 - ☐ Plankalkül was more widely used than Fortran I.

1 points

QUESTION 12

1. Pseudocode languages were quickly replaced by high-level programming languages like Fortran and COBOL. Consider the following factors, and **select** only the ones that contributed to high-level languages quickly replacing pseudocode languages. Note that incorrectly selected options will be penalised.
- ☐ Some pseudocode languages were far less writable than high-level programming languages
 - ☐ Some pseudocode language programs were expanded into machine code.
 - ☐ The computers that all pseudocode languages were designed to run on did not have hardware support for floating point calculations.
 - ☐ No pseudocode languages had any support for indexing arrays.

2 points

QUESTION 13

1. Fortran 90 was the first version of Fortran to move away from the most important programming language design concern focused on by earlier versions of Fortran. Consider the following statements, and **select** only the ones that contributed to this change of focus for Fortran 90. Note that incorrectly selected options will be penalised.
- ☐ Fortran 90 relaxed the fixed code format requirements of earlier versions of Fortran.
 - ☐ Fortran 90 added support for a `CASE` statement.
 - ☐ Fortran 90 added support for recursion.
 - ☐ Fortran 90 added support for dynamic arrays.

2 points

QUESTION 14

1. LISP was originally designed for the domain of artificial intelligence programming. Consider the following factors, and **select** only the ones that most directly contributes to the usefulness of LISP in this domain. Note that incorrectly selected options will be penalised.
- ☐ LISP supports symbolic values.
 - ☐ LISP does not support variables.
 - ☐ LISP is based on lambda calculus.
 - ☐ LISP supports the representation and processing of lists

2 points

QUESTION 15

1. ALGOL 60 made several contributions to the field of programming languages. Consider the following statements, and **select** the one that most correctly describes a contribution made by ALGOL 60.
- ☐ ALGOL 60 had well-defined I/O operations.
 - ☐ ALGOL 60 demonstrated that a programming language could be designed well for a specific hardware platform.
 - ☐ ALGOL 60 was very well suited to describing and communicating algorithms.
 - ☐ ALGOL 60 was translatable to machine code.

1 points

QUESTION 16

1. Both PL/I and ALGOL 68 aimed to support a variety of general purpose data structures, but ALGOL 68 used a better approach to achieve this. Consider the following factors, and **select** the one that most contributed to the approach used by ALGOL 68 being superior.
- ☐ ALGOL 68 combined many more data structure related constructs than PL/I.
 - ☐ ALGOL 68 was more orthogonal than PL/I
 - ☐ ALGOL 68 had data structures with a lower execution cost than those provided by PL/I.
 - ☐ ALGOL 68 supported data abstraction, while PL/I did not.

1 points

QUESTION 17

1. Consider the following statements about the Pascal and C programming languages, and **select** the one that is the most correct.
- ☐ Pascal is more readable than C.
 - ☐ Pascal is more writable than C.
 - ☐ Pascal is more reliable than C.
 - ☐ Pascal has a lower cost of writing programs than C.

1 points

QUESTION 18

1. Consider the following statements about the Java and C++ programming languages, and **select** the one that is the most correct.
- ☐ Java is less reliable than C++.
 - ☐ Java is less readable than C++.
 - ☐ Java uses the concepts of object-oriented programming more consistently than C++.
 - ☐ Java is less portable than C++.

1 points

QUESTION 19

1. Imagine you apply a function to a parameter in Scheme, as follows (assume that `aValue` is a name that has a value bound to it):

```
(doSomething aValue)
```

You want the `doSomething` function to change the value of `aValue`. Consider the following statements, and **select** the one that is the most correct.

- ☐ You can achieve this by using a `let` within the definition of the `doSomething` function.
- ☐ You can achieve this by using a `define` within the definition of the `doSomething` function.
- ☐ You can achieve this by performing an arithmetic operation on `aValue` within the definition of the `doSomething` function.
- ☐ It is impossible to achieve this in Scheme.

1 points

QUESTION 20

1. Consider the following statements about the addition operation in Scheme, and **select** the one that is the most correct.
- ☐ The addition operation is always implemented as an operator in Scheme.
 - ☐ The addition operation is always implemented as a function in Scheme.
 - ☐ The addition operation is implemented as either an operator or a function in Scheme, depending on the interpreter or compiler that is being used.
 - ☐ The addition operation is implemented as a predicate in Scheme.

1 points

QUESTION 21

1. Imagine you apply a function to a list parameter in Scheme, as follows:

```
(doSomething '(* 3 4))
```

You wish to perform the multiplication described in the list parameter. Consider the following statements, and **select** the one that is the most correct.

- ☐ The multiplication is automatically performed before the list is passed to the `doSomething` function.
- ☐ You can achieve this by providing the name of the list parameter in parentheses within the definition of the `doSomething` function.
- ☐ You can achieve this by applying the `eval` function to the list parameter within the definition of the `doSomething` function.
- ☐ It is impossible to achieve this in Scheme.

1 points

QUESTION 22

1. **Write** a Scheme function named `coneVolume`, which receives two numeric parameters. The first parameter is the radius of the circular base of a cone. The second parameter is the height of the cone. The function should yield (not print out) the volume of the cone, calculated according to the following equation:

where r is the radius of the base, and h is the height of the cylinder. The function should yield a zero result if either the radius or the height is negative. Use a `let` (not one or more `define` functions) to bind the value of the division for the name `third` that represents the first fraction in the equation, and bind a value of the division for the name `pi` that represents π .

For example, the function application

```
(coneVolume 2.8 1.7)
```

should yield a result of approximately 13.962666666666665.

Submit your program code in a file named `s99999999.scm` (where 99999999 is your student number) to the assignment submission slot labelled "Semester Test 1: Arithmetic Scheme Function", and select the "Yes" answer below once you have done so. If you do not make a submission, select the "No" answer below.

Take careful note of the following requirements:

- Your submission must be working Scheme code that will be successfully interpreted by version 8.8 of the DrRacket interpreter using the sicp language collections (this means, amongst other things, that lowercase letters should be used for all built-in function names). You may (and should) use the DrRacket interpreter to test your submission.
- You may define additional helper functions.
- Your Scheme function may only use the following built-in functions in the way they are used in the slides and textbook (failure to do so will result in a zero mark for this question):
 - Function construction: `lambda`, `define`
 - Binding: `let`
 - Arithmetic: `+`, `-`, `*`, `/`, `abs`, `sqrt`, `remainder`, `modulo`, `min`, `max`
 - Boolean values: `#t`, `#f`
 - Equality predicates: `=`, `>`, `<`, `>=`, `<=`, `even?`, `odd?`, `zero?`, `negative?`, `eqv?`, `eq?`
 - Logical predicates: `and`, `or`, `not`
 - List predicates: `list?`, `null?`
 - Conditionals: `if`, `cond`, `else`
 - Quoting: `quote`, `'`
 - List manipulation: `list`, `car`, `cdr`, `cons`
 - Input and output: `display`, `printf`, `newline`, `read`

☐ Yes

☐ No

5 points

QUESTION 23

1. Write a Scheme function named `duplicateNonMatchingValues`, which receives two parameters. The first parameter is a simple list containing only atoms. The second parameter is an atom. The function should yield (not print out) a list containing all the values in the parameter list, in their original order, with all values that do not match the second parameter duplicated.

For example, the function application

```
(duplicateNonMatchingValues '() 'A)
```

should yield an empty list because there are no atoms in the parameter list. As another example, the function application

```
(duplicateNonMatchingValues '(A D A C B) 'A)
```

should yield the list `(A D D A C C B B)` because `D`, `C`, and `B` do not match `A` in the list, and are therefore duplicated (occur twice in succession), while the two occurrences of `A` in the list are not duplicated.

Submit your program code in a file named `s999999999.scm` (where `999999999` is your student number) to the assignment submission slot labelled "Semester Test 1: List Processing Scheme Function", and select the "Yes" answer below once you have done so. If you do not make a submission, select the "No" answer below.

Take careful note of the following requirements:

- Your submission must be working Scheme code that will be successfully interpreted by version 8.8 of the DrRacket interpreter using the sicc language collections (this means, amongst other things, that lowercase letters should be used for all built-in function names). You may (and should) use the DrRacket interpreter to test your submission.
- You may define additional helper functions.
- Your Scheme function may only use the following built-in functions in the way they are used in the slides and textbook (failure to do so will result in a zero mark for this question):
 - Function construction: `lambda`, `define`
 - Binding: `let`
 - Arithmetic: `+`, `-`, `*`, `/`, `abs`, `sqrt`, `remainder`, `modulo`, `min`, `max`
 - Boolean values: `#t`, `#f`
 - Equality predicates: `=`, `>`, `<`, `>=`, `<=`, `even?`, `odd?`, `zero?`, `negative?`, `eqv?`, `eq?`
 - Logical predicates: `and`, `or`, `not`
 - List predicates: `list?`, `null?`
 - Conditionals: `if`, `cond`, `else`
 - Quoting: `quote`, `'`
 - List manipulation: `list`, `car`, `cdr`, `cons`
 - Input and output: `display`, `printf`, `newline`, `read`

☐ Yes

☐ No

5 points